Flagella-cascade investigation — changes, 2026-08-06

Reference document for every change made to the flagella assembly/regulation mechanism this session: what changed, where, why, the citation behind it, and a before/after comparison. Ordered roughly as implemented.

---

1. FliT:FliD chaperone checkpoint (real negative feedback on FlhD4C2)

Files: - `v2ecoli/processes/parca/reconstruction/ecoli/flat_overrides/complexation_reactions_added.tsv` — `FLIT-DIMER_RXN` - `v2ecoli/processes/parca/reconstruction/ecoli/flat_overrides/equilibrium_reactions.tsv` + `equilibrium_reaction_rates.tsv` — `FLIT-FLID-CPLX_RXN` - `v2ecoli/processes/flagella_flit_flhdc_checkpoint.py` — the checkpoint Step - `v2ecoli/composites/ecoli_baseline.py` — wiring into the `flagella_regulation` feature

Why: the flagella cascade (FlhD4C2 → Class II → FliA → Class III → more flagella) had no closed-loop brake. The first fix tried was a hard-coded ceiling on complete-flagellum count (rejected — see item 5). This is the real mechanism instead.

Citations: - Yamamoto S & Kutsukake K (2006) *J Bacteriol* 188:5124 — FliT exists as a homodimer, binds FliD 2:1 - Yakhnin H et al. PMC4239645 — FliT enhances FlhD4C2 ClpXP degradation catalytically ("0.02 μM FliT2 sufficiently enhanced degradation of a 100-fold molar excess of FlhC") - Tomoyasu T et al. (2003) *Mol Microbiol* 48:443 — ClpXP degrades the FlhD4C2 complex specifically, not free subunits or free FliT - Utsey B & Keener JP (2020) *PLOS Comput Biol* 16:e1007689 — fast-equilibrium reduction used for the checkpoint's math (`v = u/(1+x/K_half)`)

Before: no FliT mechanism at all; nothing tied flagellum completion back to shutting off FlhD4C2/Class II/III transcription.

After: FliT-dimer forms (complexation) → binds free FliD 1:1 (equilibrium, using the dimer as one order-1 reactant per Yamamoto & Kutsukake's 2:1 biology) → as flagella complete, demand on the shared free-FliD pool drops → equilibrium shifts toward dissociation, releasing free FliT-dimer → free FliT accelerates FlhD4C2's ClpXP degradation (catalytic, FliT itself recycled, not consumed) → less Class II/III transcription. No hard-coded ceiling anywhere; whatever limit emerges is a property of this titration dynamic.

Known caveat (flagged in code, not resolved): no literature Kd exists for FliT:FliD binding specifically, and `K_half=50` reuses the SUM-gate's K_flhDC constant for internal consistency, not as independent validation — both rate constants are estimates, not measured values.

---

2. Real assembly stoichiometry (vendored reactions were badly wrong)

Files: - `.../flat_overrides/complexation_reactions_modified.tsv` — `CPLX0-7452_RXN`, `FLAGELLAR-MOTOR-COMPLEX_RXN` - `.../flat_overrides/complexation_reactions_added.tsv` — `CPLX0-7450_RXN` (new)

Why: every vendored flagellar complexation reaction used ParCa's `null` stoichiometry default, which resolves to **-1 copy of every reactant** — correct for many ordinary complexes, badly wrong for large multi-subunit ring/rod/filament structures.

Citations & before → after (per component):

reaction	component	before	after	source
`CPLX0-7452_RXN` (flagellum)	FliC (filament)	-1	**-20,000**	PMC7696725 ("~20,000 flagellins… several micrometers long")
	FlgK (HFJ proximal)	-1	**-11**	cryo-EM structural consensus
	FlgL (HFJ distal)	-1	**-11**	cryo-EM structural consensus
	FliD (cap)	-6 (already non-null)	**-5**	pentamer consensus, Nat Commun s41467-020-16981-4
	FlgE (hook)	-120	-120 (unchanged, already correct)	—
	FLAGELLAR-MOTOR-COMPLEX	-1	-1 (unchanged, correct)	—
`FLAGELLAR-MOTOR-COMPLEX_RXN`	FliF (MS-ring)	-1	**-34**	cryo-EM C34 symmetry; corroborated independently by Chang/Sung/Hong 2025
	FlgH (L-ring)	-1	**-26**	cryo-EM C26 symmetry
	FlgI (P-ring)	-1	**-26**	cryo-EM C26 symmetry
	FliE	-1	**-6**	cryo-EM
	FlgB	-1	**-5**	cryo-EM
	FlgC	-1	**-6**	cryo-EM
	FlgF	-1	**-5**	cryo-EM
	FlgG (distal rod)	-1	**-24**	cryo-EM (older biochemical estimate was ~26)
	MotA (stator)	-1	**-55**	derived: 5:2 MotA:MotB ratio × ~11 stator units/motor (medium confidence)
	MotB (stator)	-1	**-22**	same derivation
	FliL	-1	**-2**	Maya's "Master Flagella Info" spreadsheet

reaction	component	before	after	source
	CPLX0-7450 (switch complex)	*(FliG/FliM/FliN consumed directly, no staging)*	**-1**, new staged intermediate	see below
	CPLX0-7451 (export apparatus)	*(built, never consumed — orphaned)*	**-1**	structural gap fix, real biology requires it in the assembled base
`CPLX0-7450_RXN` (new — motor switch complex)	FliG	*(didn't exist)*	**-34**	cryo-EM C34; PMC10128058
	FliM	*(didn't exist)*	**-34**	PMC10128058
	FliN	*(didn't exist)*	**-111**	PMC10128058, "Precise Measurement of the Stoichiometry of the Adaptive Bacterial Flagellar Switch"

Open discrepancy, unresolved: FliG=34 (two independent literature sources) vs. Maya's spreadsheet's value of 26 — flagged in code comments, needs her source if she has one for 26.

All original vendored stoichiometry preserved as comments in the TSV files per standing practice, not deleted.

---

3. Gillespie SSA blowup → incremental Step architecture

Files: - `v2ecoli/processes/parca/reconstruction/ecoli/dataclasses/process/complexation.py` — `RUNTIME_EXCLUDED_REACTIONS` - New Steps: `flagella_motor_switch_assembly.py`, `flagella_motor_complex_assembly.py`, `flagella_filament_nucleation.py`, `flagella_filament_elongation.py` - `v2ecoli/library/schema_types.py` — new `nascent_flagellum` unique molecule type - `v2ecoli/library/initial_conditions.py`, `.../dataclasses/state/internal_state.py` — registration

Why: once item 2's real stoichiometry was in place, ParCa hung/crashed with `"failed simulation: total propensity is NaN"`. Root-caused via direct instrumentation (macOS `sample` + `strings` on the compiled `.so`, not guessing) to Gillespie SSA's propensity calculation — a **product** of per-reactant combinatorial terms — overflowing. Confirmed empirically: a single coefficient of 60 (pre-existing `LUMAZINESYN-CPLX_RXN`) is safe; 5 simultaneous double-digit coefficients (`FLAGELLAR-MOTOR-COMPLEX_RXN`) are not, and neither is one coefficient of 20,000 (`CPLX0-7452_RXN`).

No literature citation — this is a numerical/methods finding, not a biology claim.

Before: `CPLX0-7452_RXN`, `CPLX0-7450_RXN`, `FLAGELLAR-MOTOR-COMPLEX_RXN` ran through the generic `ecoli-complexation` Gillespie process. First attempted fix — excluding them from the runtime process config alone — was **insufficient**, because ParCa's own `step_05_fit_condition.py` reads `sim_data.process.complexation` directly, bypassing the runtime-only filter.

After: excluded at the source — `Complexation.__init__` itself skips these 3 reaction IDs when building its internal reaction set (`RUNTIME_EXCLUDED_REACTIONS`), protecting both consumers. Replaced with 4 dedicated Steps doing the same real stoichiometry deterministically/ incrementally instead of stochastically. `FLIT-DIMER_RXN` (coefficient of only 2) was left in ordinary Gillespie complexation — no numerical issue at that scale.

---

4. Nucleation & elongation kinetics (real literature rate laws)

Files: `flagella_filament_nucleation.py`, `flagella_filament_elongation.py`

Why: once filament growth moved out of Gillespie (item 3), it needed an actual time-dependent growth model — not just corrected stoichiometry.

Citations: - Renault TT et al. (2017) *eLife* 6:e23136 — injection-diffusion filament growth model, `dL/dt = a/(b+L)`, `a≈26,450 subunit²/s`, `b≈575 subunits` (derived from their reported initial rate ~83-100 nm/min and characteristic length ~0.27 μm) - Chang, Sung & Hong (2025) *Biochem Biophys Reports* 42:102051 — qualitative mechanism only (excess FliF/FlhA feeds existing basal-body clusters rather than nucleating new ones); gives no rate constant - Sisti F et al. (2017) *Sci Rep* 7:41189 — nucleation rate is a **derived estimate**, back-calculated from their observed ~2-8 flagella/cell range reached over one generation (~5 flagella / ~50 min ⇒ ~1 nucleation event per ~10 min ⇒ rate ≈ 0.00167/s), not a direct citation

Before: no elongation kinetics existed — filament formation was an instantaneous, all-or-nothing conversion (the nature of a complexation reaction), incompatible with real gradual growth even before considering the numerical blowup.

After: `flagella_filament_nucleation.py` fires on a fixed ~10-minute interval, creating one `nascent_flagellum` (filament_length=0) per event if material allows. `flagella_filament_elongation.py` grows every existing instance's `filament_length` each tick per Renault's rate law, fair-sharing available free FliC across simultaneous filaments when demand exceeds supply. Completes at `filament_length ≥ 20,000` (see item 2's FliC coefficient — the two are kept consistent by construction).

Bug found and fixed same day: the first implementation computed `n_events = round(nucleation_rate * timestep)` every tick — with `rate≈0.00167/s` and a ~2s timestep, that's `round(0.0033)`, which rounds to exactly 0 forever regardless of elapsed time (confirmed via a direct 2400s test: zero nascent flagella despite ample material). Fixed by switching to a fixed-interval trigger (`next_update_time` scheduled ~1/rate seconds ahead) instead of accumulating sub-1 probability per tick.

---

5. Artificial nucleation cap — implemented, then fully removed

Files: `flagella_nucleation_cap.py` (deleted), `ecoli_baseline.py` (feature step list edited), associated test scripts/charts (deleted)

Why: tried first as a stopgap before the real FliT mechanism (item 1) existed. Removed per your explicit instruction: "i dont want the artifical cap at all so remove that and any plots, scripts associations with it" — you rejected an arbitrary hard ceiling in favor of a real, citable regulatory mechanism.

No citation — this was explicitly the "quick fix" being rejected.

Before: a Step hard-capping complete-flagellum count at a fixed number, plus its test scripts and charts.

After: fully removed, no trace in the active codebase (only a stale `.pyc` remains). No hard-coded ceiling anywhere in `flagella_regulation`; whatever limit on flagella count exists now is an emergent property of the real mechanisms (items 1, 4, 7), to be observed empirically rather than assumed.

---

6. Two division-boundary bugs fixed

6a. Opt-in features silently dropped at daughter-cell rebuild

Files: `ecoli_baseline.py` (`loader._features`), `composites/_helpers.py` (`'division'` branch), `steps/division.py` (`Division.initialize`, daughter rebuild call)

Why found: crashed with `ValueError: 'EG10322_RNA' is not in list` the first time any run crossed a division boundary. Root-caused via direct instrumentation (patched `baseline()` to print resolved `features` at call time) to `division.py`'s daughter rebuild never passing `features=` at all — mirrors a pre-existing pattern (`injected_processes`) that had already been solved for a different feature but not generalized.

No citation — engineering/framework bug, not biology.

Before: `Division` step rebuilt each daughter via `baseline(...)` with no `features=` argument. Since the opt-in feature global clears itself right after the parent's build, daughters silently reverted to plain baseline — and worse, `process_bigraph`'s structural-realize mechanism still tried to rebuild the stale step node using the Step class's bare `config_schema` defaults (unresolved raw cistron IDs), crashing outright.

After: `loader._features` threaded through `_helpers.py`'s `'division'` branch into `Division`'s config, stored on `self._features`, passed explicitly on every daughter rebuild: `baseline(..., features=self._features)`.

6b. `nascent_flagellum` duplicated (not split) at division

Files: `v2ecoli/library/division.py` — `divide_nascent_flagellum`, registered in `UNIQUE_DIVIDERS` **and** the explicit dispatch chain in `divide_cell`

Why found: flagged proactively (not asked) before running the first multi-generation test — any unique molecule type without a registered divider fell through to `divide_cell`'s catch-all, which **copies the whole array to both daughters**. For `nascent_flagellum` this is physically impossible (a single anchored structure can't become two) and would have duplicated every in-progress flagellum at every division, exponentially inflating the count across generations and invalidating any multi-gen test.

Rationale (no direct citation, mirrors existing `divide_bulk` design): real flagella are physically anchored to a fixed point on the cell envelope; division splits that envelope roughly in half, so a binomial p=0.5 per-flagellum assignment (each independently, entirely, to one daughter — not split fractionally) is the natural approximation for a peritrichous, ~randomly-distributed set of flagella — mirroring how complete flagella (an ordinary bulk molecule) already divide via `divide_bulk`'s binomial split.

Before: no divider registered → duplicated to both daughters.

After: `divide_nascent_flagellum` (binomial p=0.5, seeded from `filament_length.sum()+n_molecules`), registered in **both** the `UNIQUE_DIVIDERS` dict and the explicit if/elif dispatch chain in `divide_cell` — registering in the dict alone was confirmed insufficient (the dispatch code checks identity against known dividers explicitly, no generic fallback). Verified directly: a real division produced non-overlapping length sets in the two daughters.

---

7. FliC synthesis-rate calibration gap — found, partially fixed

Files: - `.../flat_overrides/rna_expression_adjustments.tsv` (new file — all upstream rows preserved, one row added for `EG10321_RNA`) - `.../parca_overrides.tsv` (registers the override via the `adjustments__rna_expression_adjustments` canonical key)

Why: the Class III override (`override = Y * basal_prob`, `flagella_transcription_regulation.py`) caps out **at** `basal_prob` — `Y` saturates toward 1 but never exceeds it, so the override can never exceed ParCa's population-average (non-motile-induced) fitted rate for fliC.

Method — directly measured, not estimated: ran a single cell with FliA forced high (Y≈0.98, near-saturated) and tracked total filament-subunit incorporation once the free FliC pool bottomed out and stayed there (a clean proxy for real-time synthesis rate, since consumption≈production at a near-zero buffer).

Citations: - Renault et al. 2017 (as above) — back-calculated target of ~46 molecules/s, the peak demand implied by their real growth curve - Li et al. 2014 ("Quantifying Absolute Protein Synthesis Rates…") — `translation_efficiency.tsv`'s fliC=0.55 value; investigated as a candidate lever and **ruled out** — it's a real, literature-sourced translation-efficiency number, and the bottleneck was diagnosed to be upstream (transcription-probability ceiling), not translation

	before	after
measured real-time fliC synthesis	**0.71 molecules/s** (sustained, 1800s near-zero-pool window)	**6.0 molecules/s** (sustained, 360s near-zero-pool window)
gap to Renault-implied ~46/s target	~65x	~7.6x
fliC's share of total mRNA transcription budget	~0.24%	~2.3%

Why 10x and not more (deliberate ceiling, not an oversight): all mRNA promoter probabilities share one fixed total budget (`transcript_initiation.py` 's `is_mrna` rescale) — pushing fliC further squeezes every other gene. 65x (the multiplier needed to fully close the gap) would put fliC alone at ~13.5% of the cell's entire mRNA transcription budget — judged unrealistic, would starve hundreds of co-expressed genes. 10x → ~2.3%, argued to be plausible given flagellin's known high abundance in motile cells. **Residual gap left open by design**, not further pursued per your explicit call not to touch the override architecture itself.

Mechanism note: verified this operates through ParCa Step 2 (`step_02_input_adjustments.py` 's `adjust_rna_expression`) — the same per-cistron expression-adjustment mechanism already used for 10 other genes in the base reconstruction — so the whole downstream fit (RNAP recruitment regression, ppGpp scaling, renormalization) re-derives consistently around the new target, rather than patching `basal_prob` post-hoc (which would need 5-6 correlated arrays touched to stay consistent, per `simulation_data.py` 's unused `adjust_final_expression` helper).

---

8. Framework-level fix: mass-balance tolerance (not flagella-specific)

File: `.../dataclasses/process/complexation.py` (~lines 180-214)

Why: a fixed absolute tolerance breaks down once any reaction has very large stoichiometric coefficients. Item 2's FliC correction (~20,000 copies) makes the reaction's mass-balance terms ~1e9 in magnitude, so floating-point summation alone produces an absolute residual (~1.9e-8) that's actually a *relative* error of only ~1.9e-17 — at the floor of double-precision representation — yet tripped the old fixed threshold. A prior developer had already hit this exact failure mode once before ("had to bump this up to 1e-8 because of flagella supercomplex").

No citation — pure floating-point/numerical analysis.

Before:

```
assert np.all(np.absolute(massBalanceArray) < 1e-8)
```

After:

```
atol = 1e-8
rtol = 1e-13
max_term_magnitude = np.max(np.absolute(balanceMatrix), axis=0)
tolerance = atol + rtol * max_term_magnitude
assert np.all(np.absolute(massBalanceArray) < tolerance)
```

Scale-aware: small absolute floor for ordinary reactions, plus a relative allowance (rtol=1e-13, ~1000x looser than the ~1e-17 noise actually observed, but still ~13 orders of magnitude tighter than any real stoichiometry bug would produce) proportional to each reaction's own largest term. Fixes the root cause instead of bumping the same brittle constant a third time.

---

9. Multi-generation validation

File: `workspace/investigations/flagella-cascade/studies/flagella-02-transcription-regulation/run_flit_checkpoint_multigen.py`

Why: stress-test whether items 1-8 together produce realistic, self-limiting flagella dynamics across real cell divisions — the original question this whole investigation arc was chasing.

Reference for comparison: Sisti et al. 2017 (as above) — real E. coli range of 2-8 flagella/cell, shown as a shaded band on the population chart.

Result (2hr sim, 4 cells, cache built after item 7's fix): population grew 1→4 cells over 3 divisions, all splitting `nascent_flagellum` correctly (item 6b holding up under real multi-gen load). Nucleation kept firing (30 flagella under construction by t=7,200s), but **zero completed** — longest filament reached 10,730 of the 20,000-subunit target (~halfway) and stalled. Mean flagella/cell fell 4.0→1.0 as division diluted inherited flagella faster than new ones could finish.

Open question, not yet answered: whether this self-corrects given more time/generations, or whether the residual ~7.6x fliC gap (item 7) means completions structurally can't keep pace with division-driven dilution.

Still open / not addressed this session

- FliG=34 (2 lit. sources) vs. Maya's spreadsheet's 26 (item 2)
- Whether the FlgM→FliA→Class III loop has a real closed-loop brake now that fliC supply is the binding constraint (entangled with item 7 — nothing can run away if filaments can't complete in the first place)
- Whether flagella count self-limits realistically once/if item 7's residual gap is addressed